Python Grundlagen – Kontrollstrukturen

KW39 · mit KI-Protokoll

1. if / elif / else

Was ist das? Mit if kann das Programm entscheiden, was passieren soll. Wenn die Bedingung stimmt, wird der eingerückte Teil ausgeführt. Mit elif kann ich noch eine Bedingung prüfen und else ist für alles andere.

Beispiel

```
alter = 16

if alter >= 18:
    print("Du bist volljährig.")
elif alter >= 16:
    print("Du darfst Moped fahren.")
else:
    print("Du bist noch zu jung.")
```

Ausgabe

```
Du darfst Moped fahren.
```

Erklärung: 16 ist nicht größer gleich 18, aber größer gleich 16, deshalb kommt der elif-Teil. elif kann man nur nach einem if schreiben und es wird nur geprüft, wenn die Bedingungen davor nicht gestimmt haben. Wichtig: Nach if/elif/else kommt ein Doppelpunkt und der Code darunter muss eingerückt sein.

KI-Protokoll

Prompt: *„Erklär mir if, elif und else in Python mit einem einfachen Beispiel.“*

2. for-Schleife

Was ist das? Eine for-Schleife geht der Reihe nach durch eine Liste oder einen Zahlenbereich. Man benutzt sie, wenn man weiß, wie oft etwas wiederholt werden soll.

Beispiel

```
einkauf = ["Apfel", "Brot", "Milch"]
for artikel in einkauf:
    print("Ich kaufe:", artikel)

for i in range(1, 4):
    print(i)
```

Ausgabe

```
Ich kaufe: Apfel
Ich kaufe: Brot
Ich kaufe: Milch
1
2
3
```

Erklärung: Bei jedem Durchlauf ist artikel das nächste Element aus der Liste. range(1, 4) gibt 1, 2, 3 aus, die 4 ist nicht mehr dabei.

KI-Protokoll

Prompt: *„Wie funktioniert eine for-Schleife in Python? Bitte einfaches Beispiel.“*

3. while-Schleife

Was ist das? Eine while-Schleife läuft so lange, wie die Bedingung stimmt. Man nimmt sie, wenn man nicht genau weiß, wie oft wiederholt werden muss.

Beispiel

```
countdown = 3
while countdown > 0:
    print(countdown)
    countdown -= 1
print("Start!")
```

Ausgabe

```
3
2
1
Start!
```

Erklärung: Bei jedem Durchlauf wird countdown um 1 kleiner. Bei 0 stimmt die Bedingung nicht mehr und die Schleife hört auf. Ohne countdown -= 1 würde die Schleife nie aufhören (Endlosschleife).

KI-Protokoll

Prompt: *„Was ist eine while-Schleife und was ist der Unterschied zur for-Schleife?“*

4. break

Was ist das? Mit break wird eine Schleife sofort beendet, auch wenn sie noch nicht fertig ist.

Beispiel

```
zahlen = [4, 7, 12, 3, 9]
for z in zahlen:
    if z > 10:
        print("Gefunden:", z)
        break
    print("Prüfe", z)
```

Ausgabe

```
Prüfe 4
Prüfe 7
Gefunden: 12
```

Erklärung: Sobald eine Zahl größer als 10 gefunden wird, hört die Schleife auf. 3 und 9 werden gar nicht mehr geprüft.

KI-Protokoll

Prompt: *„Was macht break in Python? Zeig mir ein kurzes Beispiel.“*

5. pass

Was ist das? pass macht einfach gar nichts. Python braucht nach einem Doppelpunkt aber immer Code, deshalb schreibt man pass als Platzhalter hin, wenn man dort (noch) nichts machen will.

Beispiel

```
def spaeter():
    pass   # mache ich später

for n in range(1, 6):
    if n % 2 == 0:
        pass
    else:
        print(n, "ist ungerade")
```

Ausgabe

```
1 ist ungerade
3 ist ungerade
5 ist ungerade
```

Erklärung: Bei geraden Zahlen passiert nichts (pass), bei ungeraden wird etwas ausgegeben. Ohne pass würde Python einen Fehler geben, weil der Block leer wäre.

KI-Protokoll

Prompt: *„Wofür braucht man pass in Python?“*

6. try / except / else / finally

Was ist das? Mit try/except kann man Fehler abfangen, damit das Programm nicht abstürzt. Der Aufbau hat vier Teile: In **try** kommt der Code, bei dem ein Fehler passieren kann. **except** wird ausgeführt, wenn ein Fehler passiert. **else** wird nur ausgeführt, wenn *kein* Fehler passiert ist. **finally** wird immer ausgeführt, egal ob ein Fehler kam oder nicht.

Beispiel

```
eingabe = "abc"
try:
    zahl = int(eingabe)
    ergebnis = 100 / zahl
except ValueError:
    print("Das ist keine Zahl!")
except ZeroDivisionError:
    print("Durch 0 geht nicht!")
else:
    print("Ergebnis:", ergebnis)
finally:
    print("Fertig.")
```

Ausgabe

```
Das ist keine Zahl!
Fertig.
```

Erklärung: "abc" kann man nicht in eine Zahl umwandeln, deshalb kommt ein ValueError und der passende except-Teil wird ausgeführt. else wird übersprungen, weil ein Fehler passiert ist, finally kommt aber trotzdem. Bei eingabe = "0" käme „Durch 0 geht nicht!“, bei eingabe = "4" käme „Ergebnis: 25.0“ (else). In beiden Fällen kommt danach noch „Fertig.“. Wichtig: else und finally sind optional, die Reihenfolge ist aber immer: try, except, else, finally.

KI-Protokoll

Prompt: *„Wie fange ich mit try und except Fehler in Python ab? Einfaches Beispiel bitte.“*
Prompt: *„Wofür braucht man else und finally bei try/except? Bitte im selben Beispiel ergänzen.“*

7. Bedingte Ausdrücke

Was ist das? Ein bedingter Ausdruck ist ein if/else in einer einzigen Zeile. Man verwendet ihn, wenn man je nach Bedingung einen von zwei Werten will.

Beispiel

```
punkte = 42
ergebnis = "bestanden" if punkte >= 50 else "nicht bestanden"
print(ergebnis)
```

Ausgabe

```
nicht bestanden
```

Erklärung: Aufbau: *Wert1 if Bedingung else Wert2*. 42 ist nicht größer gleich 50, deshalb wird der Wert nach else genommen.

KI-Protokoll

Prompt: *„Was sind bedingte Ausdrücke in Python? Erklär es einfach mit Beispiel.“*

8. match / case (Switch-Case)

Was ist das? Python hat kein klassisches switch, aber seit Python 3.10 gibt es match/case. Ein Wert wird mit mehreren Fällen verglichen und der passende Fall wird ausgeführt.

Beispiel

```
tag = 7
match tag:
    case 1:
        print("Montag")
    case 2:
        print("Dienstag")
    case 6 | 7:
        print("Wochenende!")
    case _:
        print("Unbekannt")
```

Ausgabe

```
Wochenende!
```

Erklärung: 7 passt zu case 6 | 7 (der Strich heißt „oder“). case _ ist der Standardfall, wenn nichts anderes passt. Anders als in Java braucht man kein break.

KI-Protokoll

> **Prompt:** *„Gibt es in Python ein switch-case? Wie funktioniert match case?“*

Quellen

• https://openbook.rheinwerk-verlag.de/python/05_001.html

• https://www.datacamp.com/tutorial/python-switch-case

• KI: Claude (Anthropic)